

Le jeu Find My Word

Théo Palugan
Loïc Lakshminarayanan

BUT 1 Informatique

Groupe : Vénus

Année 2025 - 2026

IUT de Villetaneuse (Sorbonne université paris 13)

Java (SAE S201)

28/04/2026 - 12/06/2026

Sommaire :

I - Analyse et organisation :	3
II - Planning :	3
III - Diagramme UML Simplifié :	4
IV - Diagramme UML Complet :	4
V - Lancement du programme :	6
VI - Choix réalisés :	7
VII - Décisions techniques :	8
VIII - Difficultés rencontrées et solutions apportées :	9

I - Analyse et organisation :

Tâches concrètes : Planification, Répartition, Jalon, Réaliser un diagramme UML simplifié, Développement progressif, Test régulier, Bonnes pratiques, Respecter les principes de la programmation orientée objet, Validation du fonctionnement, Relecture, Corrections et optimisation, Préparer les livrables, Justification des choix, Retour critique, Bilan.

Compétences à mobiliser : Connaissance en Java, Recherche d'informations, Travail d'équipe, Gestion de projet.

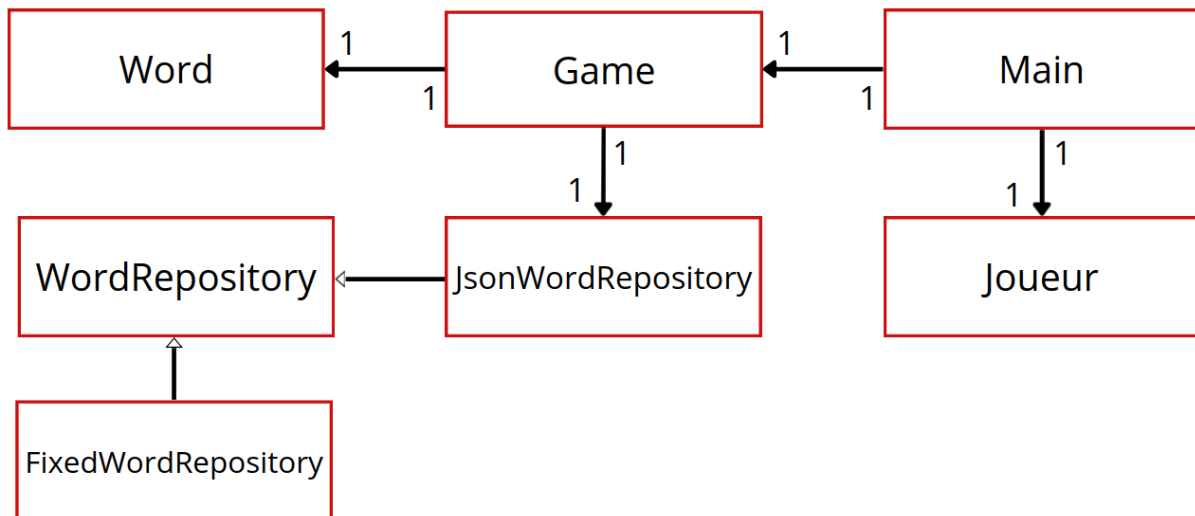
Outils et technologie : Canva, Visual Studio Code.

II - Planning :

Étant donné que la rédaction se fait à chaque séance, nous avons préféré ne pas le préciser dans le planning.

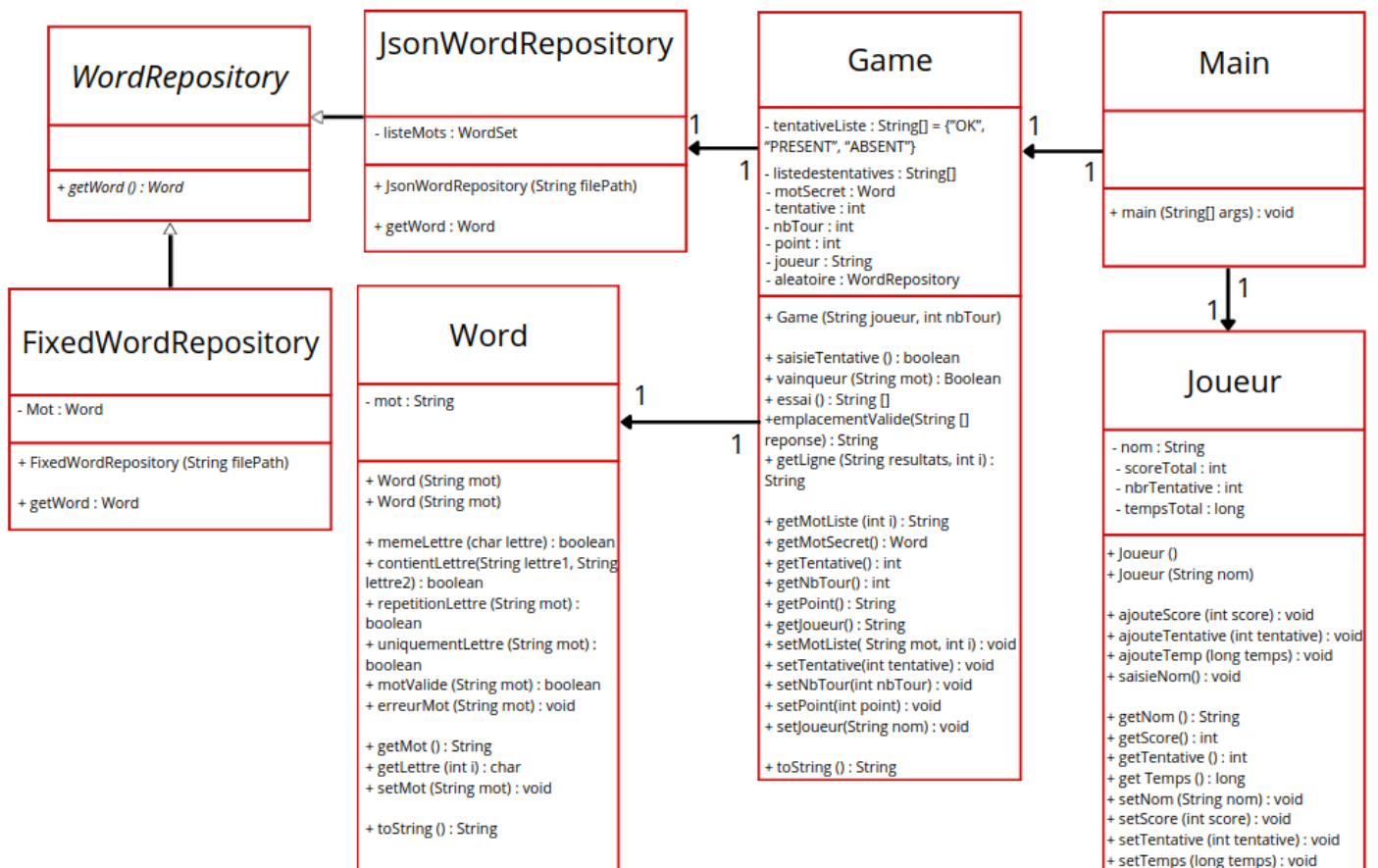
Séance n°:	Théo	Loïc
1	Lecture du sujet + planification + UML Simplifié + commencement rédaction rapport.	
2	Réflexion logique du jeu + UML complet (ajout attributs + méthodes).	UML complet + comprendre le fonctionnement de JSON en JAVA (fichier JSON word.json)
3	Création classe Main + classe Game	Création classes Word et WordRepository
4	Respect les conditions + tests	Implémenter récupération mot à l'aide du fichier JSON + validation entrée
5	Analyse des mots (OK / PRESENT / ABSENT)	Affichage console + grille de jeu
6	Boucle de jeu + conditions victoire/défaite	Tests + corrections bugs
7	Amélioration du code => lisibilité + affichage	
8	Tests complets + corrections	
10	Finalisation rapport +Vérification globale du projet	

III - Diagramme UML Simplifié :



Nous avons des associations unidirectionnelle car Main va accéder à Game et a Joueur, ce qui n'est pas le cas pour Game et Joueur qui eux ne pourront pas accéder à Main. C'est exactement le même schéma pour Game et Word, et Game et JsonWordRepository. Nous avons également fait en sorte que JsonWordRepository et FixedWordRepository soient des classes filles de WordRepository.

IV - Diagramme UML Complet :



La classe **Main** est celle qui va contenir tout le système d'interface et va permettre au joueur d'interagir avec le programme. Elle ne contient pas de méthode particulière, à part le **Main**, qui permet de lancer le programme.

La classe **Joueur** va permettre de garder les informations relatives aux différents utilisateurs que ce soit dans le mode de jeu individuel ou en multijoueur. Elle permet avec ces différentes méthodes comme : **ajouteScore**, **ajouteTentative** et **ajouteTemps** d'incrémenter des variables utiles pour les statistiques de fin de partie et le bon déroulement du jeu. Mais elle contient également la méthode **saisieNom** qui permet aux utilisateurs de rentrer leur nom ou pseudo.

La classe **Game** quant à elle va permettre au joueur d'interagir avec le programme, pour ce faire nous allons pouvoir utiliser la méthode **saisieTentative** qui va utiliser un scanner pour permettre au joueur de taper un mot dans le terminal. Si le mot est valide nous utilisons la méthode **essai** qui permet de retourner un tableau de String de ce type : {"OK", "ABSENT", "OK", "PRESENT", "ABSENT"}. Ce tableau qui permet de savoir si les lettres sont bien placées, présentes ou absentes du mot secret et qui va ensuite permettre à la méthode **emplacementValide** de retourner une phrase de ce style : "OK ABSENT OK PRESENT ABSENT". Grâce à la méthode **getLigne** nous pouvons afficher au joueur le résultat du mot essayé, si le mot secret était LIGNE et que le joueur a essayé le mot LOGER alors le résultat sur la console sera : [L][O][G][E][R] => OK ABSENT OK PRESENT ABSENT. La méthode **Vainqueur**, elle, permet de vérifier si le mot secret a été trouvé et d'attribuer les points en conséquence.

Quant à la classe **WordRepository**, c'est une classe abstraite qui sert de modèle pour les différentes manières de récupérer un mot secret. En effet, elle contient la méthode abstraite **getWord** (qui devra donc être redéfinie dans les classes qui héritent d'elle).

La classe **JsonWordRepository** hérite ensuite de **WordRepository**, elle permet de récupérer un mot aléatoire depuis le fichier JSON fourni avec la SAE (mots.json), grâce à la bibliothèque **wordset.jar**. En effet, elle est utilisée lors d'une partie normale.

La classe **FixedWordRepository** hérite également de **WordRepository**, elle permet de retourner toujours le même mot (ce qui est surtout utile pour les tests, car elle va nous permettre de connaître le mot secret en avance et donc de vérifier plus facilement si le programme fonctionne).

A l'aide de cette organisation, la classe **Game** peut utiliser un objet de type **WordRepository** sans dépendre directement de **JsonWordRepository** ou de **FixedWordRepository**. Cela permet d'utiliser le polymorphisme, car un même type général peut représenter plusieurs classes concrètes différentes.

Et pour finir la classe **Word** qui va permettre avec **memeLettre** et **contientLettre** de savoir où se situe la lettre ou si le mot la contient. **repetitionLettre** et **uniquementLettre** vont permettre à **motValide** de savoir si le mot saisi par le joueur respecte les règles du jeu et, si ce n'est pas le cas, va appeler **erreurMot** pour afficher d'où vient l'erreur.

V - Lancement du programme :

Le programme doit être lancé depuis un terminal. Pour ce programme, les fichiers nécessaires à la compilation sont :

- **Game.java**
- **Main.java**
- **Word.java**
- **WordRepository.java**
- **JsonWordRepository.java**
- **FixedWordRepository.java**
- **Joueur.java**

Le programme utilise également la bibliothèque fournie **wordset.jar**, située dans le dossier : **wordset_bundle_etudiant/jar/wordset.jar**, qui permet de charger les mots depuis le fichier JSON et fournit les classes **WordSet** et **JsonWordSet**. Par ailleurs, Le fichier de mots doit être situé dans ce dossier :

wordset_bundle_etudiant/data/mots.json

Pour compiler le programme, il faut se placer dans le dossier du projet avec le terminal, puis exécuter la commande suivante :

Linux / macOS :

```
javac -cp ".:wordset_bundle_etudiant/jar/wordset.jar" -d . Game.java Main.java  
Word.java WordRepository.java JsonWordRepository.java  
FixedWordRepository.java Joueur.java
```

Windows :

```
javac -cp ".;wordset_bundle_etudiant\jar\wordset.jar" -d . Game.java Main.java  
Word.java WordRepository.java JsonWordRepository.java  
FixedWordRepository.java Joueur.java
```

Cette commande va compiler les classes Java et créer le package **words**.
L'option **-cp** permet d'ajouter la bibliothèque **wordset.jar** au projet et l'option **-d** permet de créer automatiquement le dossier du package **words**.
Puis pour lancer le programme, il faut ensuite l'exécuter avec la commande :

Linux / macOS :

```
java -cp ".:wordset_bundle_etudiant/jar/wordset.jar" words.Main
```

Windows :

```
java -cp ".;wordset_bundle_etudiant\jar\wordset.jar" words.Main
```

Le programme démarre alors dans le terminal. Il demande d'abord de choisir le nombre de joueurs, soit 1 soit 2. Ensuite, il demande le nom du premier joueur, puis le nom du deuxième joueur si le mode deux joueurs est choisi. Le programme demande aussi le nombre de tours à jouer.

VI - Choix réalisés :

Pour l'organisation des classes, nous avons privilégié une architecture dans laquelle les différentes classes possèdent un rôle précis. Cette organisation nous a permis d'éviter les dépendances inutiles entre les différentes parties du programme. Les relations entre les classes ont été limitées à certaines interactions dès lors que nous les jugeons réellement nécessaires afin de conserver une structure claire.

Nous avons également accordé une grande importance à la séparation des responsabilités. Plutôt que de regrouper toutes les fonctionnalités en une seule classe, nous avons plutôt décidé de les répartir selon leur domaine. Les données des joueurs, les règles liées aux mots, la gestion de la partie et l'accès à la liste des mots sont gérés indépendamment du reste.

La logique du jeu a été répartie de manière à ce que les règles principales soient centralisées dans la gestion de la partie, tandis que les autres fonctionnalités restent dans les classes concernées. Ce choix nous permet de conserver un déroulement de jeu cohérent tout en évitant la duplication de méthodes.

Pour respecter les principes de la programmation orientée objet, nous avons donc dû utiliser l'encapsulation en regroupant les données et les traitements associés au sein des mêmes classes.

Nous avons décidé d'utiliser le polymorphisme pour respecter la nouvelle organisation demandée dans le sujet. Pour ce faire, nous avons transformé **WordRepository** en classe abstraite. Dans cette classe, nous définissons la méthode abstraite **getWord**, qui représente le fait de récupérer un mot secret.

En effet, deux classes vont ensuite hériter de **WordRepository** : **JsonWordRepository** et **FixedWordRepository**. La classe **JsonWordRepository** récupère un mot aléatoire depuis le fichier JSON, tandis que **FixedWordRepository** retourne un mot fixé à l'avance, ce qui nous permet d'avoir plusieurs façons de récupérer un mot (tout en gardant la même logique dans la classe **Game**).

Nous avons donc décidé de faire en sorte que la classe **Game** utilise donc un attribut de type **WordRepository**. De ce fait, elle ne dépend pas directement d'une classe précise, elle peut donc fonctionner avec **JsonWordRepository** pour une vraie partie ou avec **FixedWordRepository** pour des tests.

Pour finir nous avons choisi d'utiliser un tableau plutôt qu'une ArrayList, car le nombre maximal de tentatives est connu dès le début du programme et reste fixe tout le long de la partie. Un tableau était donc suffisant pour stocker tous les essais tout en offrant une solution simple et adaptée aux besoins du projet.

VII - Décisions techniques :

Pour ce projet, nous avons dû faire des choix techniques afin de respecter les contraintes de la SAE et de garder une bonne organisation du code.

Comme l'impose la SAE, nous avons séparé les responsabilités entre plusieurs classes.

La classe **Game** qui gère le déroulement de la partie avec les tentatives, l'analyse des mots proposés, les points, les tours et la détection de la victoire ou de la défaite. La logique principale du jeu est dans cette classe. De plus, nous avons également fait en sorte qu'une tentative invalide ne soit pas comptabilisée pour que cela respecte les règles du sujet (par exemple si le joueur entre un mot trop court, trop long, avec des caractères incorrects ou avec des lettres répétées, il doit pouvoir recommencer sans perdre un essai).

La classe **Word** représente un mot du jeu. Elle contient les méthodes liées au mot lui-même, comme la vérification de sa validité (avec l'ensemble des affichages des erreurs), la comparaison des lettres ou la détection des lettres répétées. Elle représente donc les contraintes à respecter.

Nous avons ensuite ajouté une classe **Joueur** en plus afin de représenter les informations propres à chaque joueur. Elle contient le nom du joueur, son score total, son nombre total de tentatives et son temps total. Elle permet aussi d'ajouter des points, des tentatives et du temps au fur et à mesure de la partie. Ce choix permet de ne pas mélanger les informations du joueur avec la logique principale du jeu. En effet, la classe Game reste centrée sur le déroulement d'une partie, tandis que la classe Joueur conserve les données propres à chaque participant.

Nous avons aussi séparé la récupération des mots du reste du programme grâce à **WordRepository**, **JsonWordRepository** et **FixedWordRepository**. Comme dit précédemment la classe **WordRepository** est une classe abstraite qui définit la méthode `getWord`. Cette méthode représente l'action de récupérer un mot secret, sans préciser directement d'où vient ce mot.

La classe **JsonWordRepository** hérite de **WordRepository** et récupère un mot aléatoire depuis le fichier `mots.json`, à l'aide de la bibliothèque `wordset.jar`.

La classe **FixedWordRepository** hérite aussi de **WordRepository**, mais elle retourne toujours le même mot.

Ce choix va nous permettre d'utiliser le polymorphisme. En effet, dans la classe `Game`, l'attribut utilisé pour récupérer le mot est de type `WordRepository`. Cependant, l'objet réel peut être un `JsonWordRepository` ou un `FixedWordRepository`. La classe `Game` utilise donc un type général, sans dépendre directement d'une classe précise.

De plus, nous avons utilisé un package **words** afin de regrouper les classes du projet dans un même espace. Cela rend la structure plus propre et respecte la contrainte de la SAE concernant l'utilisation de packages explicites.

Nous avons donc ajouté la possibilité de jouer à deux joueurs (Fonctionnalité optionnelle). Le programme demande le nombre de joueurs au lancement (1 ou 2), puis alterne les tours entre les deux joueurs. Chaque joueur possède son propre score, son nombre de tentatives et son temps de jeu. À la fin, le programme compare les scores et affiche le vainqueur ou une égalité.

Finalement, nous avons décidé d'ajouter la gestion de plusieurs tours et du temps de jeu. Ces fonctionnalités sont optionnelles dans le sujet, mais elles permettent d'améliorer le programme. Le nombre de tours est limité aux valeurs proposées dans le sujet : 1, 2, 3, 5, 10 ou 20. Le temps est mesuré pour chaque tour et pour l'ensemble de la partie afin d'apporter une information supplémentaire au joueur.

Nous avons donc décidé de ne pas ajouter une fonctionnalité optionnelle qui est l'historique des parties des joueurs. Il y a une raison à cela : nous ne trouvons pas cette fonctionnalité réellement utile, dans la mesure où il n'y a aucun intérêt à regarder les mots qui sont déjà apparus lors des parties précédentes.

VIII - Difficultés rencontrées et solutions apportées :

Lors de ce projet nous n'avons pas rencontré de grande difficulté la plus dure a été de penser à la structure que notre code allait avoir et devoir réfléchir au lien entre les classes, aux méthodes qui s'y trouveraient et à la cohérence de nos classes. La deuxième difficulté s'est présentée lors du codage pour le mot aléatoire car il a fallu utiliser un fichier JSON. C'était une chose que nous n'avions jamais faite mais grâce à nos recherches et à la documentation fournie nous avons fini par réussir.

Hormis ces points là, nous n'avons pas eu de gros problème car la création des méthodes était plutôt intuitive. Le fait que nous soyons libres durant le projet était aussi très agréable car nous avons le choix sur de nombreuses choses.

IX - Bilan du projet :

Dans notre projet, nous avons réalisé la totalité des fonctionnalités demandées et attendues dans le sujet.

Tout d'abord, notre programme permet de demander et d'enregistrer le nom du joueur en début de partie. Nous avons aussi ajouté la possibilité de choisir le nombre de joueurs : 1 ou 2. En mode deux joueurs, le programme demande le nom des deux joueurs et alterne les tours entre eux.

Il permet également de choisir un nombre de tours parmi ces valeurs proposées : 1, 2, 3, 5, 10 ou 20.

Ensuite, le mot secret est récupéré grâce à la classe abstraite **WordRepository**. Pour une partie normale, **JsonWordRepository** choisit un mot aléatoire dans le fichier mots.json, tandis que **FixedWordRepository** permet d'utiliser un mot fixe pour les tests. Puis, notre programme affiche une grille de jeu en mode texte. Après chaque tentative valide, la grille est mise à jour et les tentatives déjà jouées restent tout de même visibles.

En effet, un joueur peut saisir une proposition, cette proposition est ensuite vérifiée afin de déterminer si elle est valide. Une tentative est considérée comme invalide si elle ne respecte pas les contraintes du jeu, par exemple si le mot est trop court, trop long, contient des caractères non alphabétiques ou possède des lettres répétées. Puis notre programme va analyser chaque tentative valide et va afficher, pour chaque lettre le résultat correspondant : OK (la lettre est bien placée), PRESENT (la lettre est dans le mot, mais mal placée) ou ABSENT (la lettre n'est pas dans le mot). Un joueur gagne lorsque le mot est trouvé avec au maximum six tentatives. En cas de défaites ou de victoires, pour chaque cas un message de fin de partie est affiché.

Nous avons aussi ajouté la gestion du temps de jeu. Le programme affiche le temps utilisé pour chaque tour et, lorsqu'il y a plusieurs tours, le temps total de la partie. Enfin, le programme calcule les points obtenus et, en cas de plusieurs tours, affiche un bilan avec le total des points et le nombre total d'essais utilisé. En mode deux joueurs, le programme calcule séparément le score de chaque joueur. À la fin, il affiche le score du deuxième joueur et indique le vainqueur lorsque l'un des deux joueurs a un meilleur score. Si les deux scores sont identiques, le programme affiche une égalité.

Une fonctionnalité optionnelle n'a pas été réalisée: celle de la sauvegarde dans un fichier externe pour conserver les scores, les temps ou les résultats des parties précédentes.